

CHAPTER 06

상속과 다형성

01 상속과 다형성 소개

02 상속

03 다형성

04 is 키워드

05 클래스 자료형 변환

06 상속의 생성자

07 새도잉과 하이딩

08 하이딩과 오버라이딩

09 상속과 오버라이딩 제한

10 원도 폼: 원도 폼에서

상속과 다형성 활용하기

01 상속과 다형성 소개

- 클래스 기반의 클래스 지향 언어에서 가장 중요한 특징은 바로 상속(Inheritance)과 다형성(Polymorphism)
- 강아지와 고양이를 나타내는 클래스를 만들어보자

코드 7-1

Dog 클래스

/7장/Inheritance

```
01 class Dog
02 {
03     public int Age { get; set; }
04     public string Color { get; set; }
05
06     public Dog() { this.Age = 0; }
07
08     public void Eat() { Console.WriteLine("냠냠 먹습니다."); }
09     public void Sleep() { Console.WriteLine("쿨쿨 잠을 잡니다."); }
10     public void Bark() { Console.WriteLine("왈왈 짖습니다."); }
11 }
```

01 상속과 다형성 소개

- 고양이도 비슷한 방법으로 추상화

코드 7-2

Cat 클래스

/7장/Inheritance

```
01 class Cat
02 {
03     public int Age { get; set; }
04
05     public Cat() { this.Age = 0; }
06
07     public void Eat() { Console.WriteLine("냠냠 먹습니다."); }
08     public void Sleep() { Console.WriteLine("쿨쿨 잠을 잡니다."); }
09     public void Meow() { Console.WriteLine("냥냥 읊니다."); }
10 }
```

01 상속과 다형성 소개

- 여러 마리의 강아지와 고양이를 다루려면 배열 또는 리스트를 사용하는 것이 편리함
- [코드 7-3]은 Dog 클래스와 Cat 클래스를 생성하고 리스트에 넣은 뒤 각각의 메서드를 호출하는 예제

코드 7-3

Dog와 Cat 클래스의 인스턴스를 만들고 메서드 실행

/7장/Inheritance

```
01 static void Main(string[] args)
02 {
03     List<Dog> Dogs = new List<Dog>() { new Dog(), new Dog(), new Dog() };
04     List<Cat> Cats = new List<Cat>() { new Cat(), new Cat(), new Cat() };
05
06     foreach (var item in Dogs)
07     {
08         item.Eat();
09         item.Sleep();
10         item.Bark();
11     }
12
13     foreach (var item in Cats)
```

01 상속과 다형성 소개

```
14     {  
15         item.Eat();  
16         item.Sleep();  
17         item.Meow();  
18     }  
19 }
```

실행 결과

남남 먹습니다.
쿨쿨 잠을 잡니다.
왈왈 짹습니다.
남남 먹습니다.
쿨쿨 잠을 잡니다.
왈왈 짹습니다.
남남 먹습니다.
쿨쿨 잠을 잡니다.
왈왈 짹습니다.
남남 먹습니다.
쿨쿨 잠을 잡니다.
냥냥 읊니다.
남남 먹습니다.
쿨쿨 잠을 잡니다.
냥냥 읊니다.
남남 먹습니다.
쿨쿨 잠을 잡니다.
냥냥 읊니다.

01 상속과 다형성 소개

- [코드 7-1]부터 [코드 7-3]은 의미 없이 반복되는 부분이 굉장히 많음
- 코드의 반복을 줄인다는 것은 코드를 깔끔하게 만드는 것은 물론 생산성과 유지 보수의 측면에서 굉장히 중요한 작업
- C#에서 반복을 줄이기 위해 사용하는 방법이 바로 이번 7장의 주제인 **상속과 다형성**

02 상속

- 상속: 클래스 사이에 부모 자식 관계를 정의하는 작업
- 부모 클래스 하나는 자식을 여러 개 가질 수 있고, 부모 자신이 가진 것을 자식에게 물려주게 됨
- 현재 Dog 클래스와 Cat 클래스를 보면 다음과 같이 구성되어 있음

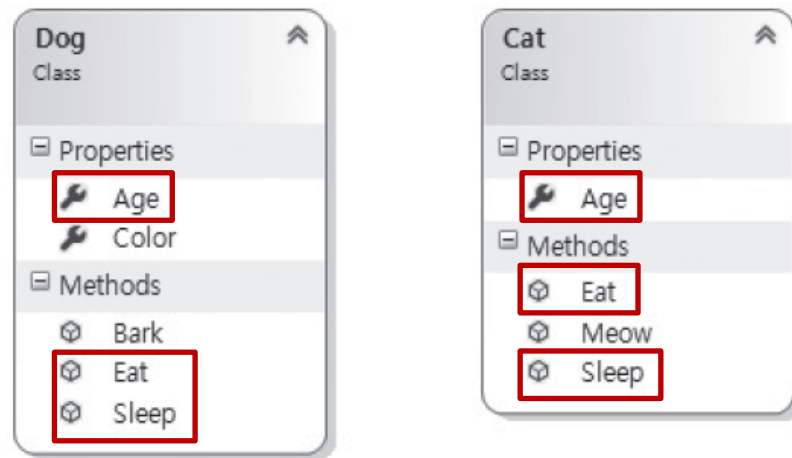


그림 7-1 분할되어 있는 클래스

02 상속

- 이 상태에서 Animal이라는 부모 클래스를 만든다면 코드를 다음과 같이 구성할 수 있게 됨

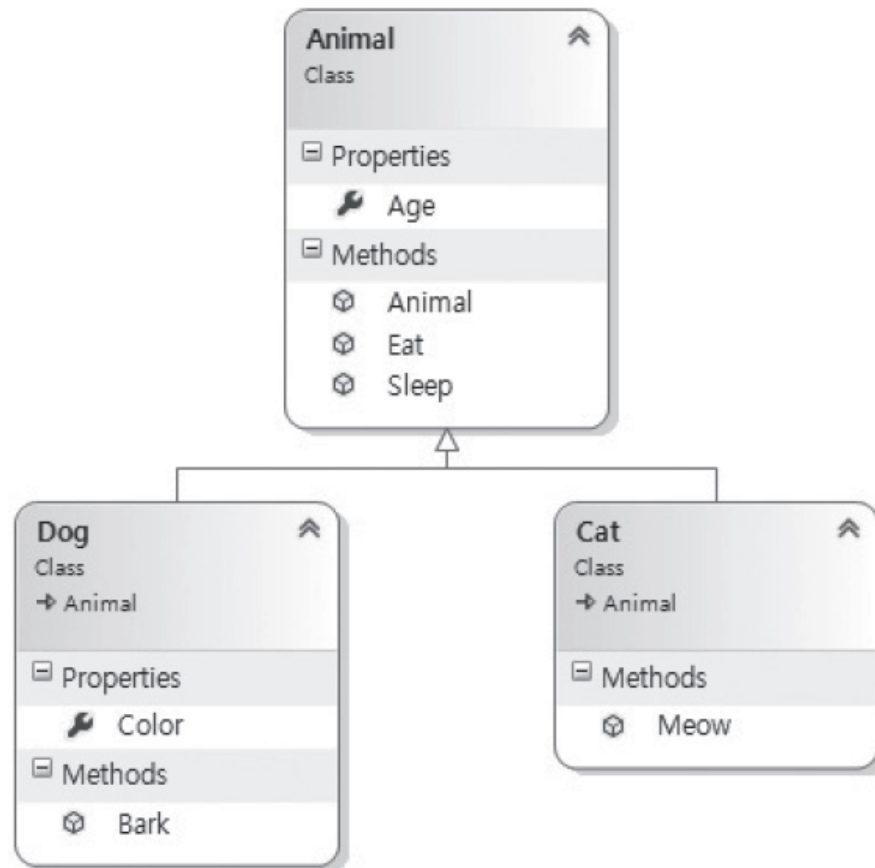


그림 7-2 상속을 사용한 구성

02 상속

- 중복되는 코드의 반복을 줄일 수 있음
- 양쪽 모두에 추가하고 싶은 멤버가 있다면 Animal 클래스만 수정하면 됨
- 이를 코드로 나타낸다면 일단 부모 클래스는 일반 클래스처럼 정의

코드 7-4

Animal 클래스

/7장/BIInheritance

```
01 class Animal
02 {
03     public int Age { get; set; }
04
05     public Animal() { this.Age = 0; }
06
07     public void Eat() { Console.WriteLine("냠냠 먹습니다."); }
08     public void Sleep() { Console.WriteLine("쿨쿨 잠을 잡니다."); }
09 }
```

02 상속

- 부모 클래스를 정의하면 자식 클래스에서 “나의 부모님은 누구입니다”라고 이야기하면 됨
- 이때는 [코드 7-5]처럼 : 기호를 사용

코드 7-5

Animal 클래스의 상속을 받는 Dog와 Cat 클래스

/7장/Inheritance

```
01 class Dog : Animal—— Animal 클래스의 상속을 받습니다.
02 {
03     public string Color { get; set; }
04
05     public void Bark() { Console.WriteLine("왈왈 짖습니다."); }
06 }
07
08 class Cat : Animal—— Animal 클래스의 상속을 받습니다.
09 {
10     public void Meow() { Console.WriteLine("냥냥 읊니다."); }
11 }
```

02 상속

- 클래스의 부모 자식 관계가 형성되면 자식 클래스는 부모 클래스의 public 또는 protected 멤버에 접근할 수 있음
- 현재 부모의 모든 멤버가 public이므로 Dog 클래스의 인스턴스를 만들면 해당 인스턴스에서 자신의 멤버는 물론 부모의 멤버에 모두 접근할 수 있음

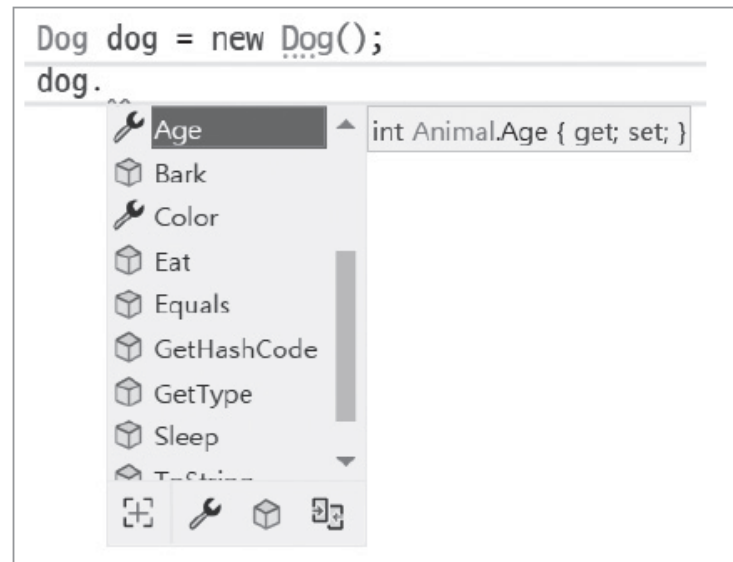


그림 7-3 Dog 클래스에서 접근할 수 있는 멤버 목록

코드 7-8

코드 중복

/7장/Inheritance

```
01 static void Main(string[] args)
02 {
03     List<Dog> Dogs = new List<Dog>{ new Dog(), new Dog(), new Dog() };
04     List<Cat> Cats = new List<Cat>{ new Cat(), new Cat(), new Cat() };
05
06     foreach (var item in Dogs)
07     {
08         item.Eat();
09         item.Sleep();
10         item.Bark();
11     }
12
13     foreach (var item in Cats)
14     {
15         item.Eat();
16         item.Sleep();
17         item.Meow();
18     }
19 }
```

비슷한 코드 중복

비슷한 코드 중복

03 다형성

- 여기에서 동물의 종류를 더 추가하려면 리스트를 다음과 같이 여러 개 만들고, 반복문도 그만큼 더 사용해야 할까?

```
List<Dog> Dogs = new List<Dog>();  
List<Cat> Cats = new List<Cat>();  
List<Snake> Lizards = new List<Snake>();  
List<Tiger> Elepents = new List<Tiger>();  
List<Lion> Lions = new List<Lion>();
```

- 이럴 때 코드 중복을 해결할 수 있는 것이 다형성
- **다형성: 하나의 클래스가 여러 형태로 변환될 수 있는 성질**

03 다형성

```
Animal dog = new Dog();  
Animal cat = new Cat();
```

- 변수 **dog**는 외관상으로는 자료형이 **Animal**이지만 실제 내부에는 **Dog**가 들어있음
- 외관상으로는 **Animal** 클래스이므로 사용할 수 있는 멤버는 **Animal** 클래스의 멤버뿐

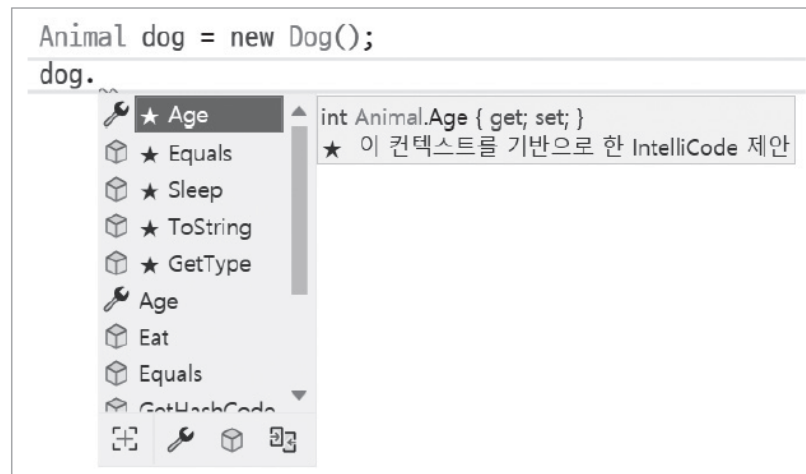


그림 7-5 부모로 위장한 자식은 부모의 멤버만 사용 가능

03 다형성

- 이렇게 부모로 위장하면 하나의 Animal 배열 또는 리스트에 여러 Dog 클래스와 Cat 클래스를 넣을 수 있음
- 따라서 [코드 7-9]처럼 코드의 중복을 제거할 수 있게 됨

코드 7-9

다형성을 사용한 코드 중복 해결

/7장/ClInheritance

```
01 static void Main(string[] args)
02 {
03     List<Animal> Animals = new List<Animal>() — 하나의 리스트를 사용합니다.
04     {
05         new Dog(), new Cat(), new Cat(), new Dog(),
06         new Dog(), new Cat(), new Dog(), new Dog()
07     };
08
09     foreach (var item in Animals)
10     {
11         item.Eat(); — 하나의 반복문을 사용합니다.
12         item.Sleep();
13     }
14 }
```

실행 결과

남남 먹습니다.
쿨쿨 잠을 잡니다.
남남 먹습니다.
쿨쿨 잠을 잡니다.
남남 먹습니다.
쿨쿨 잠을 잡니다.
남남 먹습니다.
쿨쿨 잠을 잡니다.
남남 먹습니다.
쿨쿨 잠을 잡니다.
남남 먹습니다.
쿨쿨 잠을 잡니다.

04 is 키워드

- 자식 클래스에 있는 메서드를 호출하려면 자식 클래스로 자료형을 변환해야 함
- 그럼 item이 Dog 클래스인지 Cat 클래스인지는 어떻게 구분할까?
- C#은 특정한 클래스가 어떤 클래스인지 확인하는 is 키워드를 제공

객체 is 클래스

- 이 객체가 특정한 클래스라면 true를 반환함

04 is 키워드

- 현재 코드에서 클래스를 구분할 때는 다음과 같은 코드를 사용

코드 7-14 is 키워드

/7장/DInheritance

```
01 static void Main(string[] args)
02 {
03     List<Animal> Animals = new List<Animal>()
04     {
05         new Dog(), new Dog(), new Cat(), new Cat()
06     };
07     foreach (var item in Animals)
08     {
09         item.Eat();
10         item.Sleep();
11
12         if (item is Dog) { } —— 만약 변수 item이 Dog 클래스라면
13         if (item is Cat) { } —— 만약 변수 item이 Cat 클래스라면
14     }
15 }
```

04 is 키워드

- 추가적으로 다형성의 개념이 적용되므로 item이 Dog 클래스의 클래스라면, 이는 Animal 클래스의 클래스이기도 함
- 최종적으로 올라가면
Object 클래스의 클래스라고도 할 수 있음

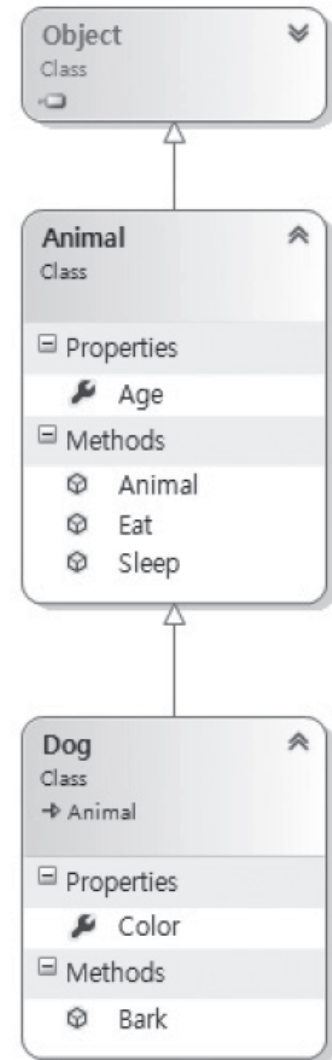


그림 7-8 Dog 클래스의 상속 관계

04 is 키워드

- 변수 item이 Dog 클래스라면 다음과 같은 경우 모두 true를 출력하게 됨

```
item is Dog  
item is Animal  
item is Object
```

05 클래스 자료형 변환

■ 일반적인 자료형 변환

- 일반적인 자료형 변환은 다음 구문을 사용

(클래스) 변수

- is 키워드를 사용한 부분에

다음과 같이 자료형 변환과 메서드 호출 부분을 추가

코드 7-15

일반적인 자료형 변환

/7장/DInheritance

```
01 foreach (var item in Animals)
02 {
03     item.Eat();
04     item.Sleep();
05
06     if (item is Dog) { ((Dog)item).Bark(); }
07     if (item is Cat) { ((Cat)item).Meow(); }
08 }
```

■ 일반적인 자료형 변환

- 코드를 실행하면 다음과 같이 item 클래스가 Dog라면 Bark() 메서드를, Cat이라면 Meow() 메서드를 호출하게 됨

실행 결과

```
남남 먹습니다.  
쿨쿨 잠을 잡니다.  
왈왈 짖습니다.  
남남 먹습니다.  
쿨쿨 잠을 잡니다.  
왈왈 짖습니다.  
남남 먹습니다.  
쿨쿨 잠을 잡니다.  
냥냥 읊니다.  
남남 먹습니다.  
쿨쿨 잠을 잡니다.  
냥냥 읊니다.
```

05 클래스 자료형 변환

■ as 키워드로 자료형 변환

- C#은 as라는 키워드를 사용해서 자료형 변환을 할 수 있음
- as 키워드를 사용해 자료형을 변환하면 변환에 실패해도 예외가 발생하지는 않고 null(아무것도 아닌 값)을 넣게 됨
- 다음과 같은 구문을 사용

변수 as 클래스

- 자료형 변환 부분을 as 키워드로 변경

코드 7-16

as 키워드를 사용한 자료형 변환

/7장/DInheritance

```
01 if (item is Dog) { (item as Dog).Bark(); }  
02 if (item is Cat) { (item as Cat).Meow(); }
```

05 클래스 자료형 변환

■ as 키워드로 자료형 변환

- 이런 경우에 as 키워드를 사용할 필요는 없음
- 사전에 is 키워드를 사용해서 어떤 클래스의 클래스인지 구분했으므로 item as Dog 또는 item as Cat이 null로 나올 가능성이 전혀 없기 때문
- as 키워드를 활용하면 [코드 7-17]과 같이 코드를 작성하는 것이 더 일반적

코드 7-17 as 키워드를 사용하는 경우의 일반적인 형태

/7장/DInheritance

```
01 foreach (var item in Animals)
02 {
03     item.Eat();
04     item.Sleep();
05
06     var dog = item as Dog;
07     if (dog != null) { dog.Bark(); }
08
09     var cat = item as Cat;
10     if (cat != null) { cat.Meow(); }
11 }
```

06 상속의 생성자

- 생성자는 인스턴스를 초기화할 때 사용
- 자식 인스턴스를 생성하면 부모가 가지고 있는 멤버 초기화를 위해 부모 생성자도 자동으로 호출됨

코드 7-18 상속했을 때 기본적인 생성자 호출 순서

/7장/ConstructorSequences

```
01 class Program
02 {
03     class Parent
04     {
05         public Parent()
06         {
07             Console.WriteLine("부모 생성자");
08         }
09     }
10
11     class Child : Parent
12     {
13         public Child()
14         {
15             Console.WriteLine("자식 생성자");
16         }
17     }
```


06 상속의 생성자

```
18
19     static void Main(string[] args)
20     {
21         Child child = new Child(); — 자식 인스턴스를 생성합니다.
22     }
23 }
```

- 코드를 실행하면 다음처럼 부모 생성자가 자동으로 먼저 호출됨
- 그리고 이어서 자식 생성자가 호출됨

실행 결과

부모 생성자

자식 생성자

06 상속의 생성자

- 만약 부모 생성자 호출을 명시적으로 지정하고 싶을 때는 [코드 7-19]처럼 base 키워드를 사용
- 생성자 메서드 뒤에 콜론을 입력하고 base()를 입력

코드 7-19 base 키워드를 사용한 생성자 지정(1)

/7장/ConstructorSequences

```
01 class Program
02 {
03     class Parent
04     {
05         public Parent() { Console.WriteLine("부모 생성자"); }
06     }
07
08     class Child : Parent
09     {
10         public Child() : base() —— base 키워드를 사용합니다.
11         {
12             Console.WriteLine("자식 생성자");
13         }
14     }
15
16     static void Main(string[] args)
17     {
18         Child child = new Child();
19     }
20 }
```

06 상속의 생성자

- 만약 매개변수가 있는 메서드를 호출하고 싶을 때는 [코드 7-20]처럼 함

코드 7-20 base 키워드를 사용한 생성자 지정(2)

/7장/ConstructorSequences

```
01 class Program
02 {
03     class Parent
04     {
05         public Parent() { Console.WriteLine("Parent()"); }
06         public Parent(int param) { Console.WriteLine("Parent(int param)"); }
07         public Parent(string param) { Console.WriteLine("Parent(string param)"); }
08     }
09
10     class Child : Parent
11     {
12         public Child() : base(10) ————— Parent(int param) 부모 생성자를 호출합니다.
13         {
14             Console.WriteLine("Child() : base(10)");
15         }
16
17         public Child(string input) : base(input) ————— Parent(string param) 부모 생성자를 호출합니다.
18         {
19             Console.WriteLine("Child(string input) : base(input)");
20         }
21     }
```

06 상속의 생성자

- 만약 매개변수가 있는 메서드를 호출하고 싶을 때는 [코드 7-20]처럼 함

```
22
23     static void Main(string[] args)
24     {
25         Child childA = new Child();
26         Child childB = new Child("string");
27     }
28 }
```

- 코드를 실행하면 base() 형태로 호출한 부모 생성자가 호출됨

실행 결과

```
Parent(int param)
Child() : base(10)
Parent(string param)
Child(string input) : base(input)
```

07 새도잉과 하이딩

- 이름 재사용: 여러 변수에 같은 이름을 사용하면 이름 사이에 이름 충돌이 발생하게 되는 것
 - 오버로딩(Overloading)
 - 오버라이딩(Overriding)
 - 새도잉(Shadowing)
 - 하이딩(Hiding)
 - 업스큐어링(Obscuring)

07 새도잉과 하이딩

■ 새도잉

코드 7-22 새도잉

/7장/ShadowAndHide

```
01 class Program
02 {
03     public static int number = 10;
04
05     static void Main(string[] args)
06     {
07         int number = 20;
08
09         Console.WriteLine(number);
10     }
```

static 메서드 내부에서 사용할 수 있게 static 변수로 만들었습니다.

실행 결과

20

- 변수의 이름이 겹칠 때 자신과 가장 가까운 변수를 사용하게 되어 있음
- 즉, 메서드 내부의 변수 number를 출력하게 됨
- 새도잉(Shadowing): 특정한 영역에서 이름이 겹쳐서 다른 변수를 가림

■ 하이딩

- 하이딩(Hiding): 부모 클래스와 자식 클래스 사이에 동일한 이름으로 멤버를 만들

코드 7-23 변수 하이딩

/7장/ShadowAndHide

```
01 class Program
02 {
03     class Parent
04     {
05         public int variable = 273;
06     }
07
08     class Child : Parent
09     {
10         public string variable = "shadowing";
11     }
12 }
```

■ 하이딩

```
13     static void Main(string[] args)
14     {
15         Child child = new Child();
16         Console.WriteLine(child.variable);
17     }
18 }
```

실행 결과

shadowing

- 이전과 마찬가지로 자신과 가장 가까운 것을 사용하게 됨
- 하이딩을 하게 되면 정상적인 상속을 막아버릴 수 있게 됨

■ 하이딩

- 부모에 있는 int 자료형의 변수를 사용하고 싶다면?
- 부모로 자료형을 변환하고 사용하면 됨

코드 7-24

숨겨진 멤버를 찾는 방법

/7장/ShadowAndHide

```
01 static void Main(string[] args)
02 {
03     Child child = new Child();
04     Console.WriteLine(((Parent)child).variable);
05 }
```

■ 하이딩

■ 메서드를 하이딩해보기

코드 7-25

메서드 하이딩

/7장/ShadowAndHide

```
01 class Program
02 {
03     class Parent
04     {
05         public void Method()
06         {
07             Console.WriteLine("부모의 메서드");
08         }
09     }
10 }
```

■ 하이딩

```
11     class Child : Parent
12     {
13         public void Method()
14         {
15             Console.WriteLine("자식의 메서드");
16         }
17     }
18
19     static void Main(string[] args)
20     {
21         Child child = new Child();
22         child.Method();
23         ((Parent)child).Method();
24     }
25 }
```

■ 하이딩

- 실행은 정상적으로 되는데, 개발 환경을 보면 다음과 같은 경고가 뜬

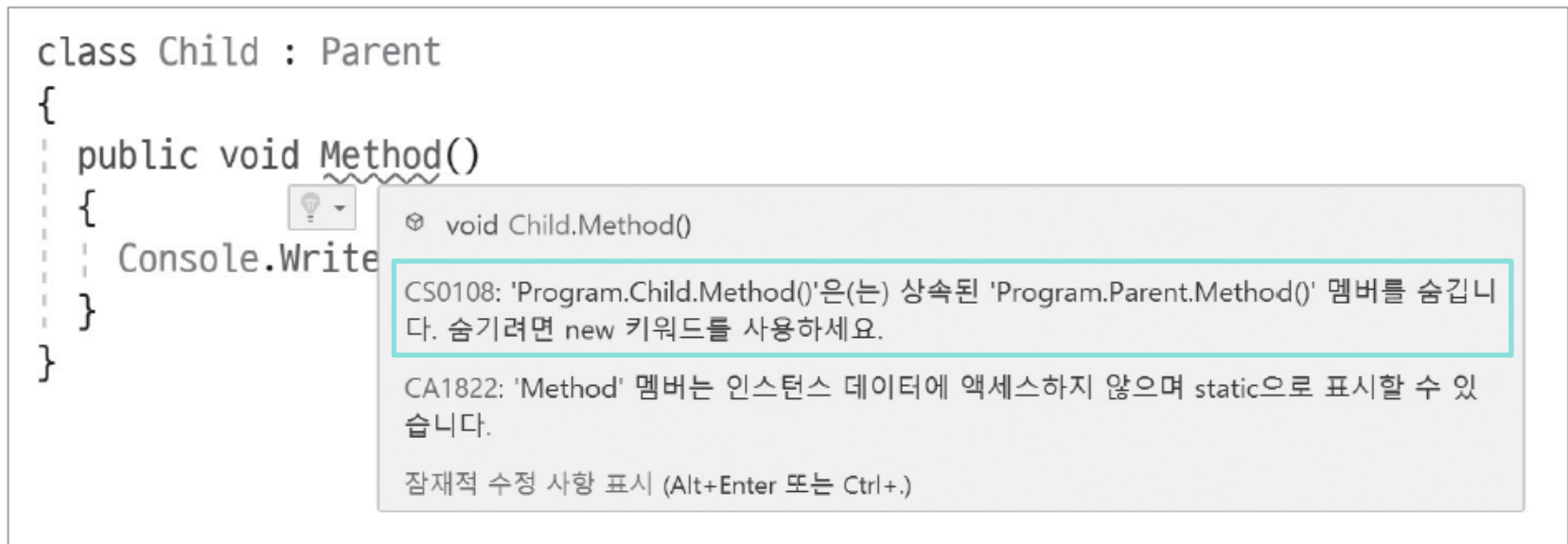


그림 7-9 경고 메시지

■ 하이딩

- 'Child.Method()은(는) 상속된 Parent.Method() 멤버를 숨깁니다.'
 - Method() 메서드가 하이딩되었음
- '숨기려면 new 키워드를 사용하세요.'
 - 메서드는 변수와 다르게 충돌이 발생할 때 하이딩할지 오버라이딩할지를 결정할 수 있음

```
public new void Method()  
{  
    Console.WriteLine("자식의 메서드");  
}
```

하이딩합니다.

```
public override void Method()  
{  
    Console.WriteLine("자식의 메서드");  
}
```

오버라이딩합니다.

08 하이딩과 오버라이딩

- C#은 부모 클래스와 자식 클래스 멤버의 이름을 동일하게 작성할 때 하이딩 또는 오버라이딩되므로 이를 명확하게 구분해야 함
- 하이딩은 부모 클래스와 자식 클래스에 동일한 이름으로 멤버를 만들 때 발생함
- 오버라이딩은 부모 클래스에 있는 메서드를 자식 클래스에서 다시 구현하는 것을 의미함

■ new 메서드

- 부모 클래스의 멤버와 자식 클래스의 멤버가 서로 이름이 같으면 하이딩됨
- 하이딩을 명확하게 표시하고자 메서드 이름 앞에 new 키워드를 붙임

코드 7-26

new 메서드를 사용한 하이딩

/7장/NewMethods

```
01 class Program
02 {
03     class Parent
04     {
05         public int variable = 273;
06         public void Method()
07         {
08             Console.WriteLine("부모의 메서드");
```

08 하이딩과 오버라이딩

■ new 메서드

```
09     }
10 }
11
12 class Child : Parent
13 {
14     public new string variable = "hiding";
15     public new void Method()
16     {
17         Console.WriteLine("자식의 메서드");
18     }
19 }
20
21 static void Main(string[] args)
22 {
23     Child child = new Child();
24     child.Method();
25     ((Parent)child).Method();
26 }
27 }
```





■ virtual과 override 메서드

- 오버라이딩을 간단하게 표현하면 '부모의 메서드를 덮어씌운다'고 할 수 있음

코드 7-27

virtual과 override 메서드를 사용한 오버라이딩

/7장/OverrideMethods

```
01 class Program
02 {
03     class Parent
04     {
05         public virtual void Method()
06     {
```

부모의 메서드에 virtual 키워드를 적용합니다.

08 하이딩과 오버라이딩

■ virtual과 override 메서드

- 이전의 new 키워드를 사용한 하이딩과 전혀 다른 결과가 나오는 것을 확인
- 오버라이딩하면 클래스형을 어떻게 변환해도 자식에서 다시 정의한 메서드가 호출됨

```
07         Console.WriteLine("부모의 메서드");
08     }
09 }
10
11 class Child : Parent
12 {
13     public override void Method()
14     {
15         Console.WriteLine("자식의 메서드");
16     }
17 }
18
19 static void Main(string[] args)
20 {
21     Child child = new Child();
22     child.Method();
23     ((Parent)child).Method();
24 }
25 }
```

자식의 메서드에 override 키워드를 적용합니다.

실행 결과

자식의 메서드
자식의 메서드

08 하이딩과 오버라이딩

■ 활용

- 왜 하이딩과 오버라이딩을 하는 것일까?

코드 7-28 하이딩

/7장/UsageOfHidding

```
01 class Program
02 {
03     class Animal
04     {
05         public void Eat()
06         {
07             Console.WriteLine("냠냠 먹습니다.");
08         }
09     }
10
11     class Dog : Animal
12     {
13         public void Eat()
14         {
15             Console.WriteLine("강아지 사료를 먹습니다.");
16         }
17     }
18
19     class Cat : Animal
20     {
21         public void Eat()
22         {
23             Console.WriteLine("고양이 사료를 먹습니다.");
24         }
25     }
26 }
```

같은 이름을 재사용했습니다.

08 하이딩과 오버라이딩

■ 활용

- 이 코드는 하이딩을 사용하고 있고, 따로 new 키워드를 적지 않았으므로 경고가 발생
- 하이딩했으므로 부모의 메서드만 실행하게 되는 것

```
27     static void Main(string[] args)
28     {
29         List<Animal> Animals = new List<Animal>()
30         {
31             new Dog(), new Cat(), new Cat(), new Dog(),
32             new Dog(), new Cat(), new Dog(), new Dog()
33         };
34
35         foreach (var item in Animals)
36         {
37             item.Eat();
38         }
39     }
40 }
```

실행 결과

냠냠 먹습니다.
냠냠 먹습니다.
냠냠 먹습니다.
냠냠 먹습니다.
냠냠 먹습니다.
냠냠 먹습니다.
냠냠 먹습니다.
냠냠 먹습니다.

08 하이딩과 오버라이딩

■ 활용

- 강아지는 강아지 사료를 먹고 고양이는 고양이 사료를 먹게 하고 싶다면?
오버라이딩 하면 됨

코드 7-29

오버라이딩

/7장/UsageOfOverriding

```
01 class Animal
02 {
03     public virtual void Eat()
04     {
05         Console.WriteLine("냠냠 먹습니다.");
06     }
07 }
08
09 class Dog : Animal
10 {
11     public override void Eat()
12     {
13         Console.WriteLine("강아지 사료를 먹습니다.");
14     }
15 }
16
17 class Cat : Animal
18 {
19     public override void Eat()
```

오버라이딩합니다

08 하이딩과 오버라이딩

■ 활용

- 이렇게 코드를 작성하면 다음과 같이 자식들이 가지고 있는 메서드를 사용하게 됨

```
20    {  
21        Console.WriteLine("고양이 사료를 먹습니다.");  
22    }  
23 }
```

실행 결과

강아지 사료를 먹습니다.
고양이 사료를 먹습니다.
고양이 사료를 먹습니다.
강아지 사료를 먹습니다.
강아지 사료를 먹습니다.
고양이 사료를 먹습니다.
강아지 사료를 먹습니다.
강아지 사료를 먹습니다.

■ 활용

- new 키워드를 사용한 하이딩은 그냥 부모의 것을 실행하고 싶을 때 사용
- [코드7-29]에서 Dog 클래스의 Eat() 메서드를 하이딩으로 바꿔보기

코드 7-30

new 키워드를 사용하는 경우

/7장/UsageOfNew

```
01 class Animal
02 {
03     public virtual void Eat()
04     {
05         Console.WriteLine("냠냠 먹습니다.");
06     }
07 }
08
09 class Dog : Animal
```

08 하이딩과 오버라이딩

■ 활용

```
10 {  
11     public new void Eat()  
12     {  
13         Console.WriteLine("강아지 사료를 먹습니다.");  
14     }  
15 }  
16  
17 class Cat : Animal  
18 {  
19     public override void Eat()  
20     {  
21         Console.WriteLine("고양이 사료를 먹습니다.");  
22     }  
23 }
```

실행 결과

남남 먹습니다.
고양이 사료를 먹습니다.
고양이 사료를 먹습니다.
남남 먹습니다.
남남 먹습니다.
고양이 사료를 먹습니다.
남남 먹습니다.
남남 먹습니다.

■ sealed 키워드

- 클래스와 메서드 앞에 적용하는 키워드
- 클래스에 적용하면 절대 상속하지 말라는 의미가 되고, 메서드에 적용하면 더 이상 오버라이딩하지 말라는 의미

■ sealed 키워드

■ 상속 제한

- 클래스 앞에 sealed 키워드를 붙일 때는 [코드 7-31]의 Parent 클래스처럼 함
- sealed 키워드를 붙였으므로 절대 상속하지 말라는 의미임에도 상속을 해서 오류가 발생

코드 7-31

sealed 클래스 오류

/7장/Sealed

```
01 class Program
02 {
03     sealed class Parent
04     {
```

sealed 클래스로 선언했습니다.

09 상속과 오버라이딩 제한

■ sealed 키워드

■ 상속 제한

```
05     public void Test() { }
06     }
07
08     class Child : Parent
09     {
10         public void Test() { }
11     }
12
13     static void Main(string[] args)
14     {
15         Parent parent = new Parent();
16         Child child = new Child();
17
18         parent.Test();
19         child.Test();
20     }
21 }
```

여기서 오류가 발생합니다.

```
class Child : Parent
{
    public void Test()
}
```

class Sealed.A.Parent

CS0509: 'Child': sealed 형식 'Parent'에서 파생될 수 없습니다.

그림 7-10 sealed 클래스 오류

■ sealed 키워드

■ 메서드 오버라이딩 제한

- sealed 키워드를 메서드 앞에 붙일 때는 원래 virtual 키워드가 붙어 오버라이딩할 수 있는 메서드를 어느 지점 이후로 못 하게 함

09 상속과 오버라이딩 제한

■ sealed 키워드

- 아래 코드도 sealed 키워드가 붙으면 더 이상 해당 메서드를 오버라이딩할 수 없어 오류 발생

코드 7-32 sealed 메서드 오류

```
01 class Parent
02 {
03     public virtual void Test() { }
04 }
05
06 class Child : Parent
07 {
08     sealed public override void Test() { }
09 }
10
11 class GrandChild : Child
12 {
13     public override void Test() { } — 여기서 오류가 발생합니다.
14 }
```

```
class GrandChild : Child
{
    public override void Test() { }
}
```

void GrandChild.Test()

CS0239: 'GrandChild.Test()': 상속된 'Child.Test()' 멤버는 봉인되어 있으므로 재정의할 수 없습니다.

그림 7-11 sealed 메서드 오류

■ abstract 키워드

- abstract 키워드는 sealed 키워드와 반대로 무조건 상속해서 쓰라는 의미 또는 이 메서드는 반드시 오버라이딩해달라는 의미로 사용
- 상속
 - abstract 키워드를 클래스에 적용하면 반드시 상속해서 쓰라는 의미
 - 해당 클래스 자체는 인스턴스를 만들어 사용할 수 없게 됨

■ abstract 키워드

- 아래 코드처럼 abstract 키워드를 적용하면 해당 클래스는 인스턴스로 만들 수 없음

코드 7-33 abstract 클래스 오류

/7장/Abstract

```
01 class Program
02 {
03     abstract class Parent
04     {
05         public void Test() { }
06     }
07
08     class Child : Parent
09     {
10         public void Test() { }
11     }
12
13     static void Main(string[] args)
14     {
15         Parent parent = new Parent();
16         Child child = new Child();
17
18         parent.Test();
19         child.Test();
20     }
21 }
```

abstract 클래스로 선언했습니다.

여기서 오류가 발생합니다.

```
Parent parent = new Parent();
```

class Abstract.Parent

CS0144: 추상 형식 또는 인터페이스 'Parent'의 인스턴스를 만들 수 없습니다.

그림 7-12 abstract 클래스 오류

■ abstract 키워드

■ 메서드 오버라이딩

- abstract 키워드를 메서드에 적용하면 이 메서드는 반드시 오버라이딩해달라는 의미
- abstract 키워드를 메서드에 적용할 때는 반드시 클래스에도 abstract 키워드를 적용해야 함
- abstract 메서드가 되면 중괄호{ }를 사용하지 않고 곧바로 세미콜론을 찍음

■ abstract 키워드

■ 메서드 오버라이딩

코드 7-34 abstract 메서드

/7장/Abstract

```
01 abstract class Parent
02 {
03     public abstract void Test();
04 }
05
06 class Child : Parent
07 {
08
09 }
```

abstract 메서드를 선언하려면 반드시 abstract 클래스가 되어야 합니다.

abstract 메서드로 선언했습니다.

여기에서 오류가 발생합니다.

```
class Child : Parent
{
    ...
}
```

class ClassBasic.Program.Child

CS0534: 'Program.Child'은(는) 상속된 추상 멤버 'Program.Parent.Test()'을(를) 구현하지 않습니다.

잠재적 수정 사항 표시 (Alt+Enter 또는 Ctrl+.)

그림 7-13 abstract 메서드 오류

■ abstract 키워드

■ 메서드 오버라이딩

- 이러한 abstract 메서드의 오류를 해결할 때는 다음과 같이 함

코드 7-35

abstract 메서드와 관련된 오류 해결

/7장/Abstract

```
01 abstract class Parent
02 {
03     public abstract void Test();
04 }
05
06 class Child : Parent
07 {
08     public override void Test() { }
09 }
```

override 키워드를 사용해 오버라이딩해야 합니다.

10 윈도 폼: 윈도 폼에서 상속과 다형성 활용하기

■ 윈도 폼에서의 상속과 다형성

■ 메서드 오버라이딩

- 이러한 abstract 메서드의 오류를 해결할 때는 다음과 같이 함

코드 7-35

abstract 메서드와 관련된 오류 해결

/7장/Abstract

```
01 abstract class Parent
02 {
03     public abstract void Test();
04 }
05
06 class Child : Parent
07 {
08     public override void Test() { }
09 }
```

override 키워드를 사용해 오버라이딩해야 합니다.

10 윈도 폼: 윈도 폼에서 상속과 다형성 활용하기

■ 윈도 폼에서의 상속과 다형성

- 다형성에 관해서는 이전에 버튼을 화면에 추가하고자 Controls.Add() 메서드의 매개변수에 버튼을 넣어주었음
- 그런데 Controls.Add() 메서드의 매개변수에 원래 넣을 수 있는 것은 Control 클래스의 인스턴스

```
public Form1()
{
    InitializeComponent();
    Controls.Add()
```

```
void Control.ControlCollection.Add(Control value)
지정된 컨트롤을 컨트롤 컬렉션에 추가합니다.
value: 컨트롤 컬렉션에 추가할 Control입니다.
```

그림 7-15 Controls.Add() 메서드의 매개변수

10 윈도 폼: 윈도 폼에서 상속과 다형성 활용하기

■ 윈도 폼에서의 상속과 다형성

- Button 클래스의 인스턴스를 넣을 수 있었던 것은 다형성 때문

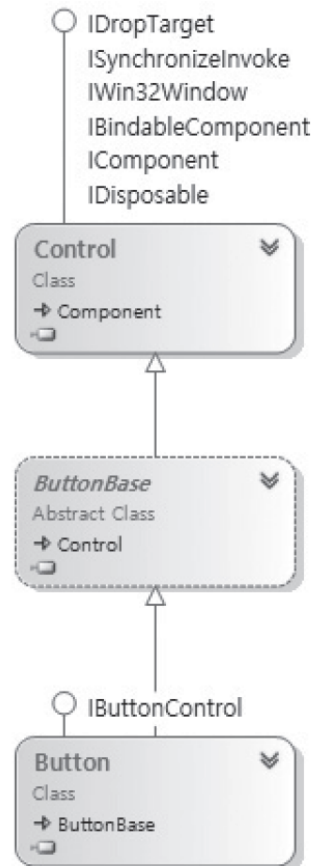


그림 7-16 Button 클래스의 상속 관계

10 윈도 폼: 윈도 폼에서 상속과 다형성 활용하기

■ 윈도 폼에서의 상속과 다형성

- 프레임워크를 만지다가 잘 모르는 내용이 있다면 하나하나 뜯어보아야 함
- 예를 들어 위와 같이 Control을 넣으라고 했는데 왜 Button을 넣는 건지 의문이 들면 직접 Button 클래스에서 마우스 오른쪽 버튼을 눌러 [정의로 이동]을 살펴보고 상속 관계를 따라가보기

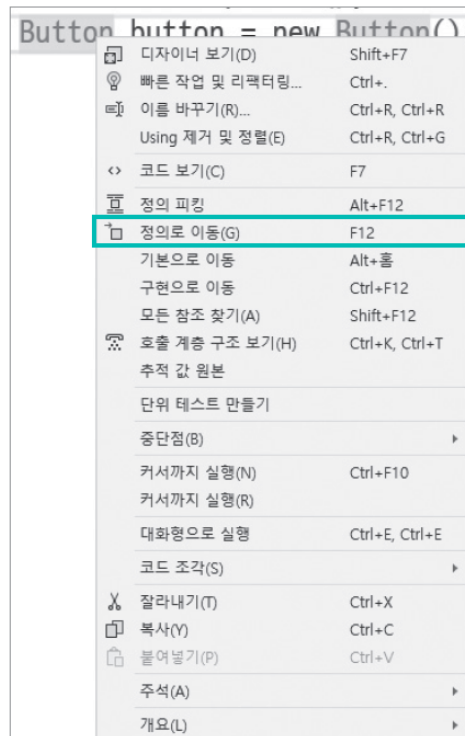


그림 7-17 정의로 이동

10 윈도 폼: 윈도 폼에서 상속과 다형성 활용하기

■ 메시지 상자

- 간단한 메시지 상자 (MessageBox 클래스)를 활용해 새로운 화면을 띄우는 방법을 알아보기
- 버튼 하나만 올려두어 화면을 다음과 같이 구성

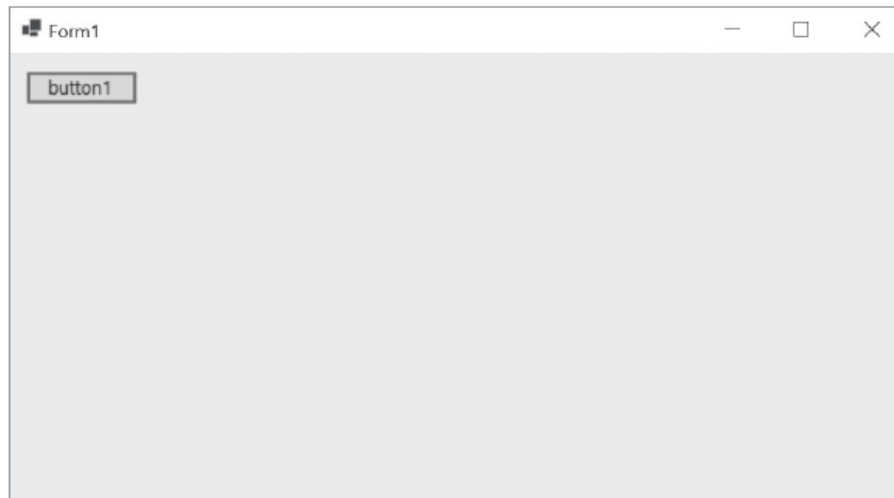


그림 7-18 화면 디자인

10 윈도 폼: 윈도 폼에서 상속과 다형성 활용하기

■ 메시지 상자

- 이어서 버튼을 더블클릭하거나 속성에서 Click 이벤트 오른쪽의 공간을 더블클릭해서 이벤트 메서드를 연결

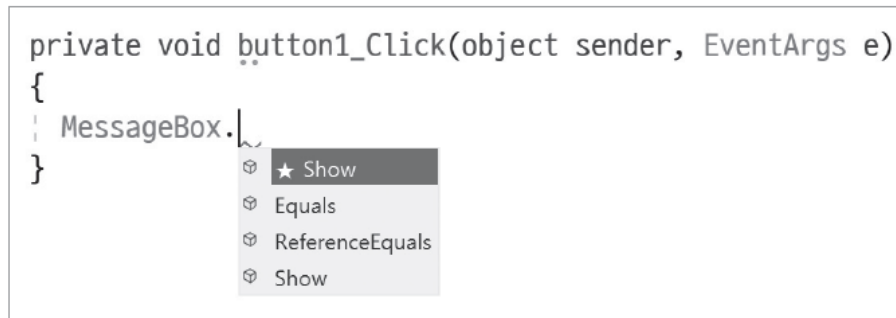


그림 7-19 MessageBox 클래스의 메서드

- static 메서드인 `Show( )`는 20가지가 넘게 오버로딩이 되어 있지만 그 중 세가지만 사용해볼 것

10 윈도 폼: 윈도 폼에서 상속과 다형성 활용하기

■ 메시지 상자

코드 7-37 MessageBox 클래스의 Show() 메서드

```
01 private void button1_Click(object sender, EventArgs e)
02 {
03     MessageBox.Show("내용");
04     MessageBox.Show("내용", "제목");
05     MessageBox.Show("내용", "제목", MessageBoxButtons.RetryCancel);
06 }
```

- 세 번째 매개변수에 넣은 MessageBoxButtons는 다음과 같은 요소들이 있고, 이 종류에 따라 다른 모양의 메시지 상자가 만들어짐

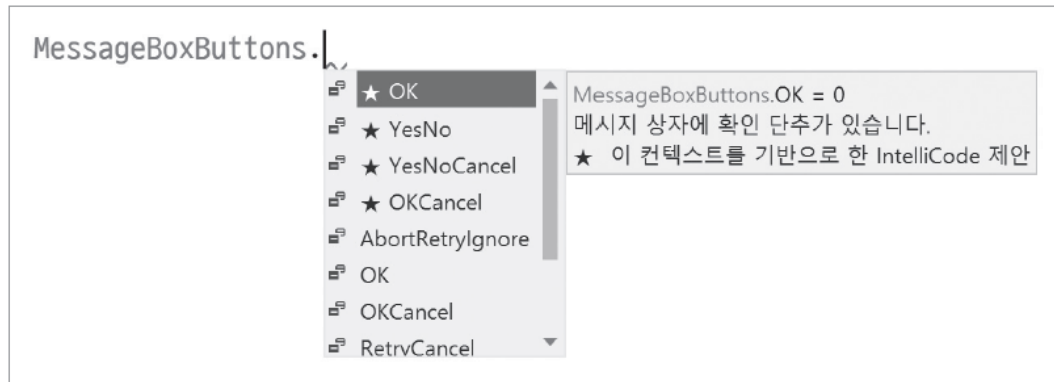


그림 7-20 MessageBoxButtons 열거자

10 윈도 폼: 윈도 폼에서 상속과 다형성 활용하기

■ 메시지 상자

- 프로젝트를 실행하고 버튼을 누르면 다음과 같은 세 종류의 메시지 상자가 차례대로 출력됨

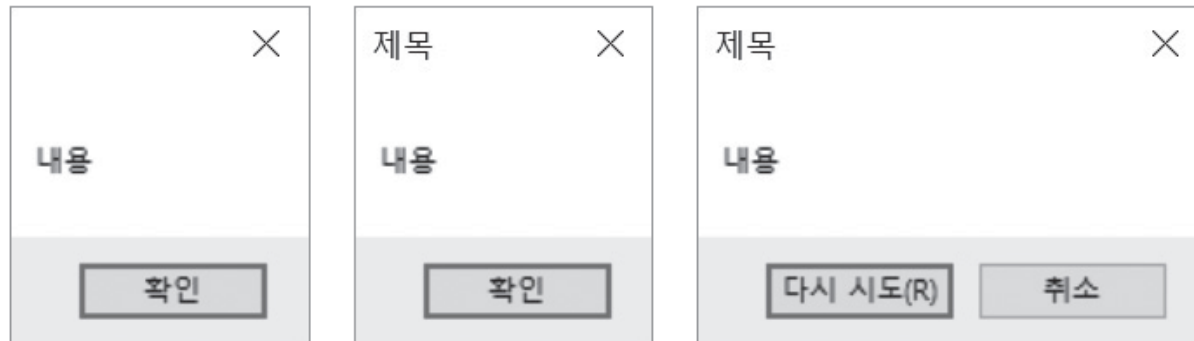


그림 7-21 메시지 상자

10 윈도 폼: 윈도 폼에서 상속과 다형성 활용하기

■ 메시지 상자

- 메시지 상자에 버튼을 누르면 Show() 메서드의 반환으로 어떤 버튼이 눌렸는지 알려줌

코드 7-38

메시지 상자 활용

```
01 private void button1_Click(object sender, EventArgs e)
02 {
03     MessageBox.Show("내용");
04     MessageBox.Show("내용", "제목");
05
06     DialogResult result;
07     do
08     {
09         result = MessageBox.Show("내용", "제목", MessageBoxButtons.RetryCancel);
10     } while (result == DialogResult.Retry);
11 }
```

적어도 한 번은 실행하게 만들고자 do while 반복문을 사용했습니다.

10 윈도 폼: 윈도 폼에서 상속과 다형성 활용하기

■ 간단한 형태의 폼 추가

- 조금 더 복잡한 형태로 새로운 화면을 띄우려면?
- Form이라는 클래스를 상속받으면 됨
- 이전과 마찬가지로의 형태로 Form1 클래스를 디자인

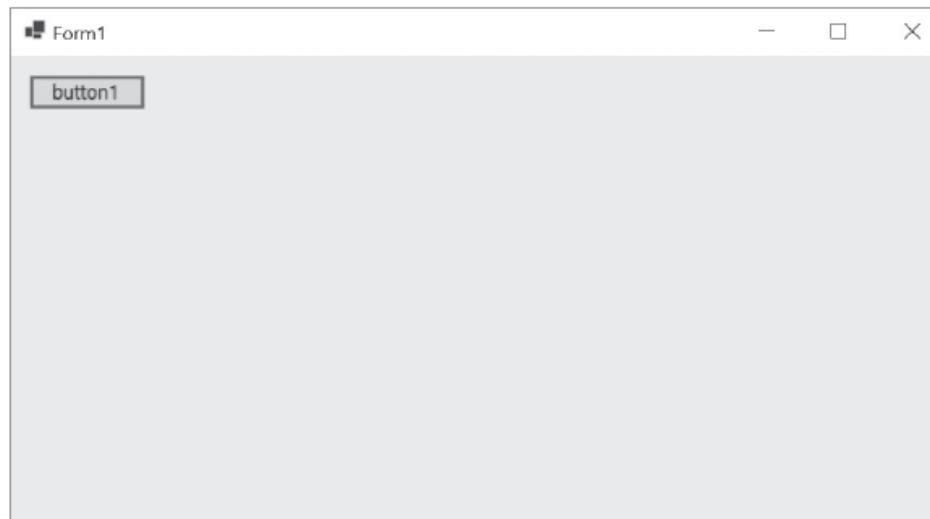


그림 7-22 화면 구성

10 윈도 폼: 윈도 폼에서 상속과 다형성 활용하기

■ 간단한 형태의 폼 추가

- 상속을 사용한 폼 생성
 - 클래스를 하나 만들고 Form 클래스의 상속을 받기

코드 7-40

Form 클래스 상속

```
01 public partial class Form1 : Form
02 {
03     class CustomForm : Form———— Form 클래스를 상속받습니다.
04     {
05
06     }
07
08     public Form1()
09     {
10         InitializeComponent();
11     }
12 }
```

10 윈도 폼: 윈도 폼에서 상속과 다형성 활용하기

■ 간단한 형태의 폼 추가

- 상속을 사용한 폼 생성
 - 새로 생성한 CustomForm 클래스에 생성자를 만들고 코드를 입력하면 속성을 지정하고, 요소를 생성할 수 있음
 - 다음 코드는 폼의 Text 속성을 지정해서 폼에 제목을 지정

코드 7-41

새로 생성한 CustomForm 클래스의 모양 지정

```
01 class CustomForm : Form
02 {
03     public CustomForm()
04     {
05         Text = "제목 글자";
06     }
07 }
```

10 윈도 폼: 윈도 폼에서 상속과 다형성 활용하기

■ 간단한 형태의 폼 추가

■ 모달리스

- 상속을 사용해서 만든 폼은 Show() 메서드로 화면에 출력할 수 있음
- 버튼에 이벤트를 연결하고 CustomForm 클래스의 인스턴스를 생성한 뒤 Show()를 호출

코드 7-42 Show() 메서드

```
01 public partial class Form1 : Form
02 {
03     class CustomForm : Form
04     {
05
06     }
07
08     public Form1()
09     {
10         InitializeComponent();
11     }
12
13     private void button1_Click(object sender, EventArgs e)
14     {
15         CustomForm form = new CustomForm();
16         form.Show(); ————— Form 클래스를 상속받습니다.
17     }
18 }
```

10 윈도 폼: 윈도 폼에서 상속과 다형성 활용하기

■ 간단한 형태의 폼 추가

■ 모달리스

- 코드를 실행하고 버튼을 누르면 [그림 7-23]처럼 새로운 폼이 뜬

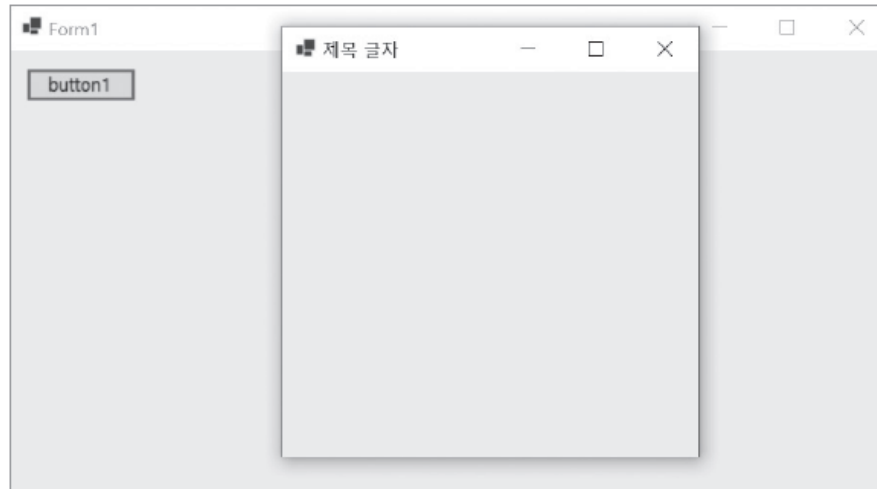


그림 7-23 Show() 메서드의 동작

- 새로운 폼을 띄웠을 때 기존의 폼을 조작할 수 있음
- 모달리스(Modeless): 새로운 화면이 열려도 기존 화면을 조작할 수 있음

10 윈도 폼: 윈도 폼에서 상속과 다형성 활용하기

■ 간단한 형태의 폼 추가

■ 모달(Modal)

- 모달리스와 반대로 새로운 화면을 띄웠을 때 기존의 화면을 조작할 수 없음
- 윈도 폼에서 Show() 메서드를 ShowDialog() 메서드로 바꿔주기만 하면 모달이 됨

코드 7-43

ShowDialog() 메서드

```
01 private void button1_Click(object sender, EventArgs e)
02 {
03     CustomForm form = new CustomForm();
04     form.ShowDialog(); — 모달 대화상자를 생성합니다.
05 }
```

10 윈도 폼: 윈도 폼에서 상속과 다형성 활용하기

■ 일반적인 형태의 폼 추가

- 디자인 화면과 함께 폼을 생성하고 싶을 때는 프로젝트에서 마우스 오른쪽 버튼을 누르고 [추가]-[양식(Windows Form)]을 선택

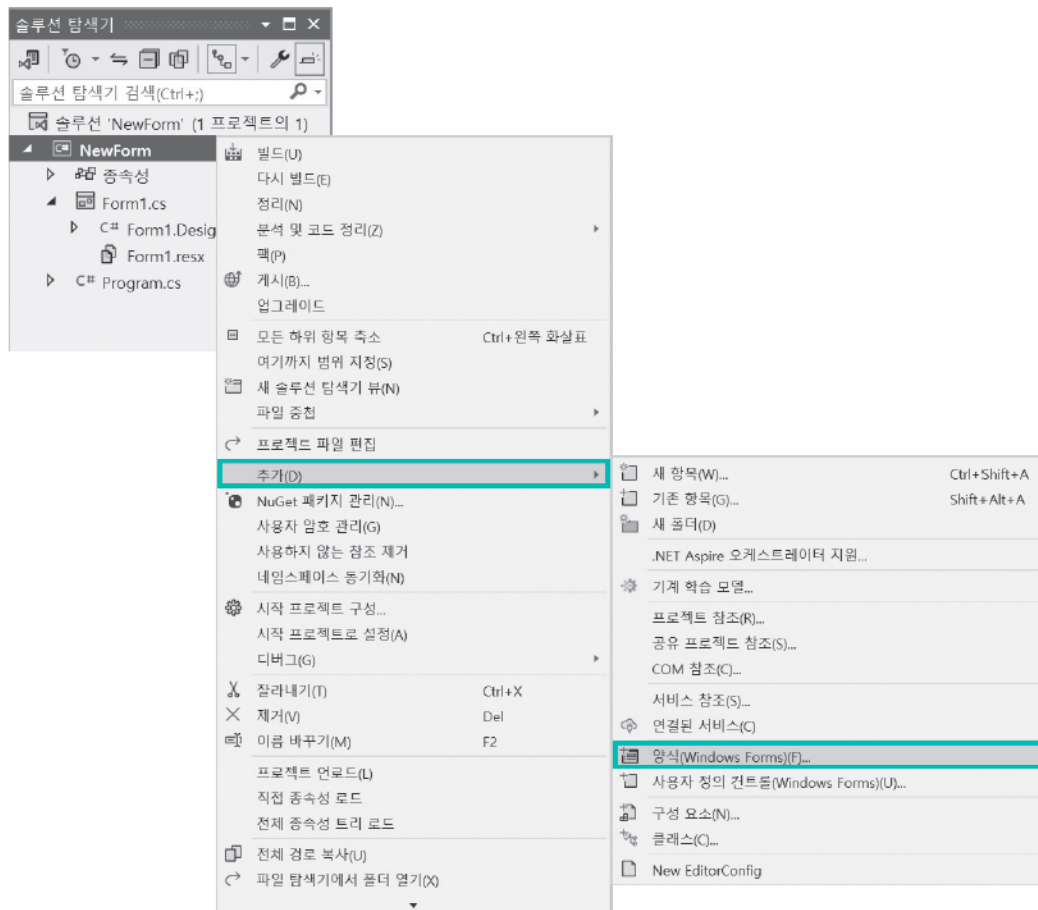


그림 7-24 [추가] - [양식(Windows Form)]으로 새로운 폼 생성

10 윈도 폼: 윈도 폼에서 상속과 다형성 활용하기

■ 일반적인 형태의 폼 추가

- 새로운 폼의 이름은 Form2로 지정
- 이렇게 폼을 생성하면 Form2.cs, Form2.Designer.cs 파일이 함께 생성됨
- 디자인 화면에서 드래그를 통해 요소를 화면에 올려놓을 수 있음